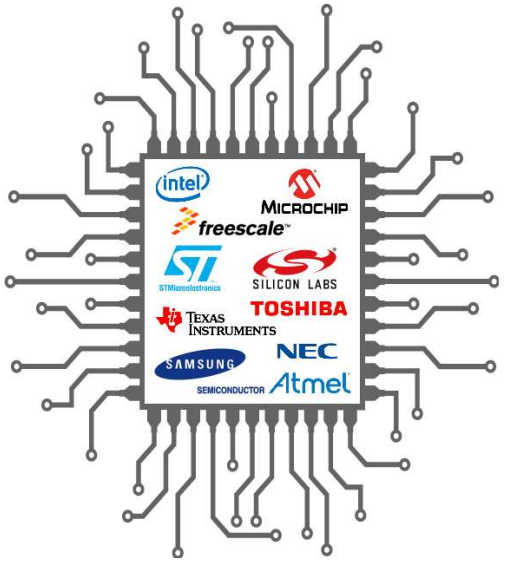
Timing and Counting



Thareendhra Keerthi Wijayasiriwardhane

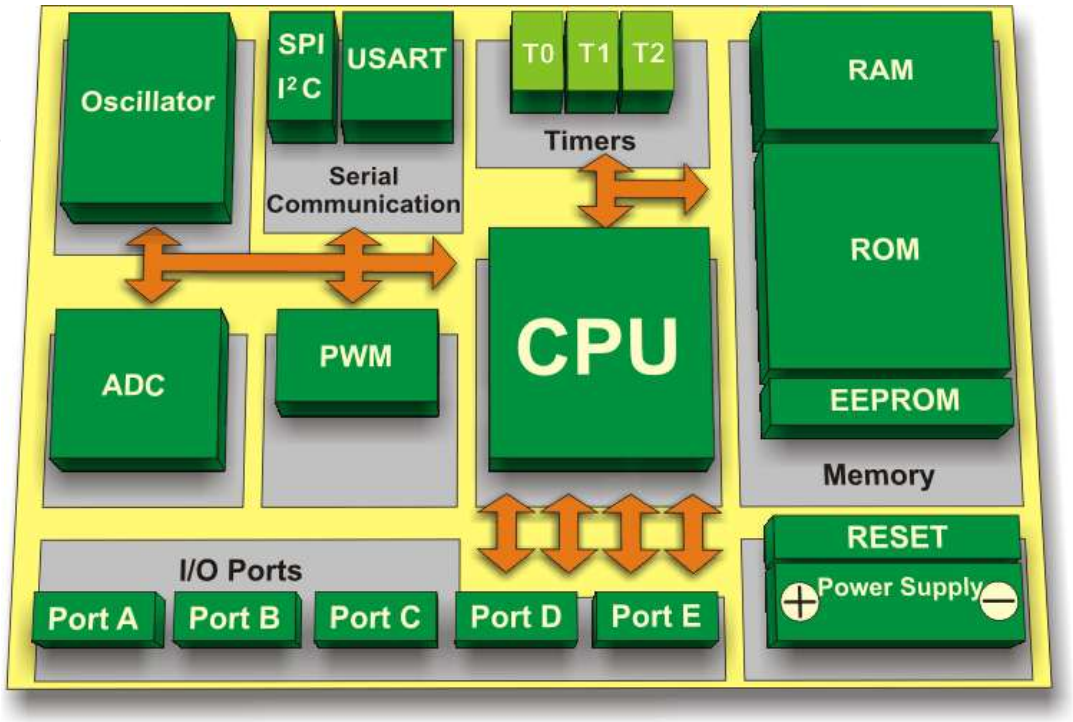
Department of Industrial Management
Faculty of Science
University of Kelaniya
thareen@kln.ac.lk

Learning Outcomes

- On completion of this session, you should be able to
  - Explain the need of Timers/Counters
  - Describe how Timers measure time and Counters count events
  - Explain the functionality of Timer/Counter registers
  - Construct a microcontroller-based system and program it to count an event
  - Construct a microcontroller-based system and program it to measure time

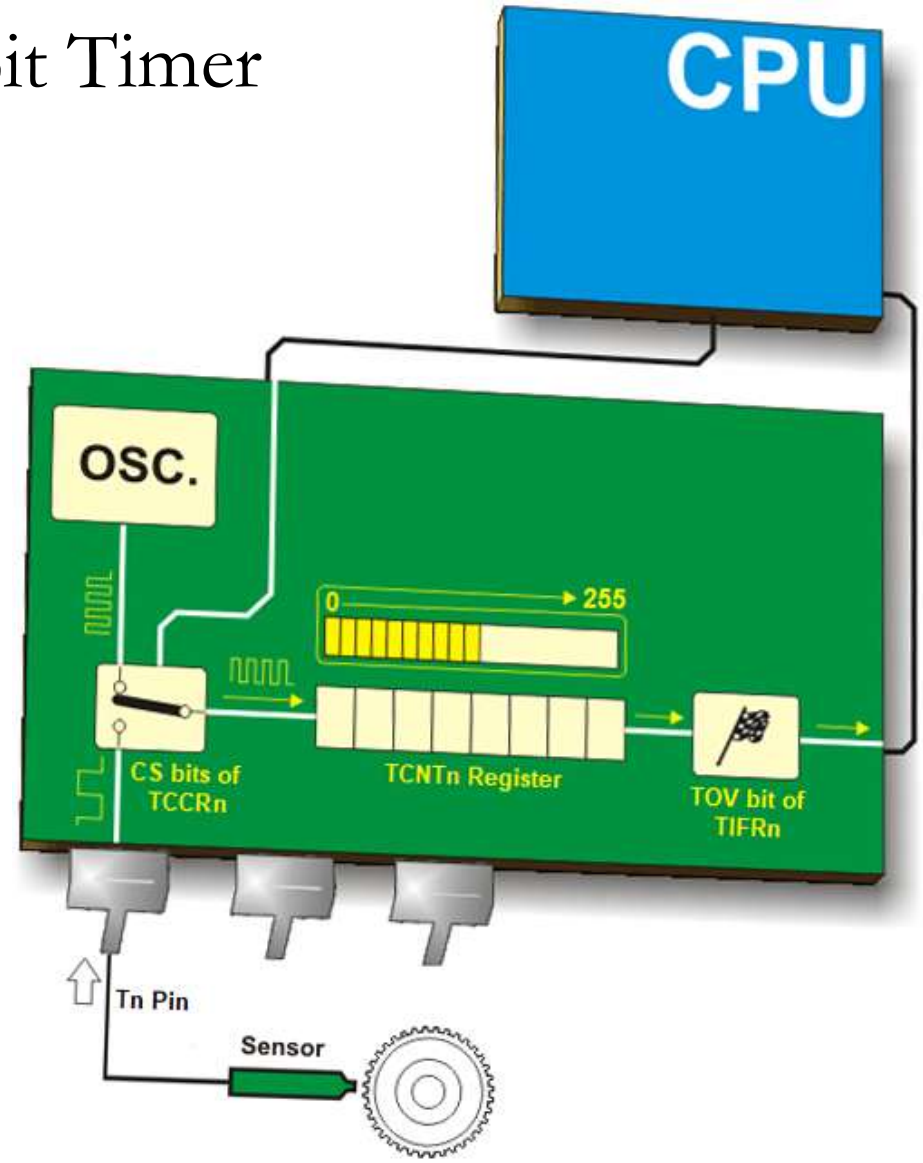
Timer

- Increments on each clock cycle when oscillator is connected
- As frequency is known, using cycle count, time elapsed can be determined
- Operates as a Counter, when an external pulse source is connected
  - Used to count events
- What's the need?

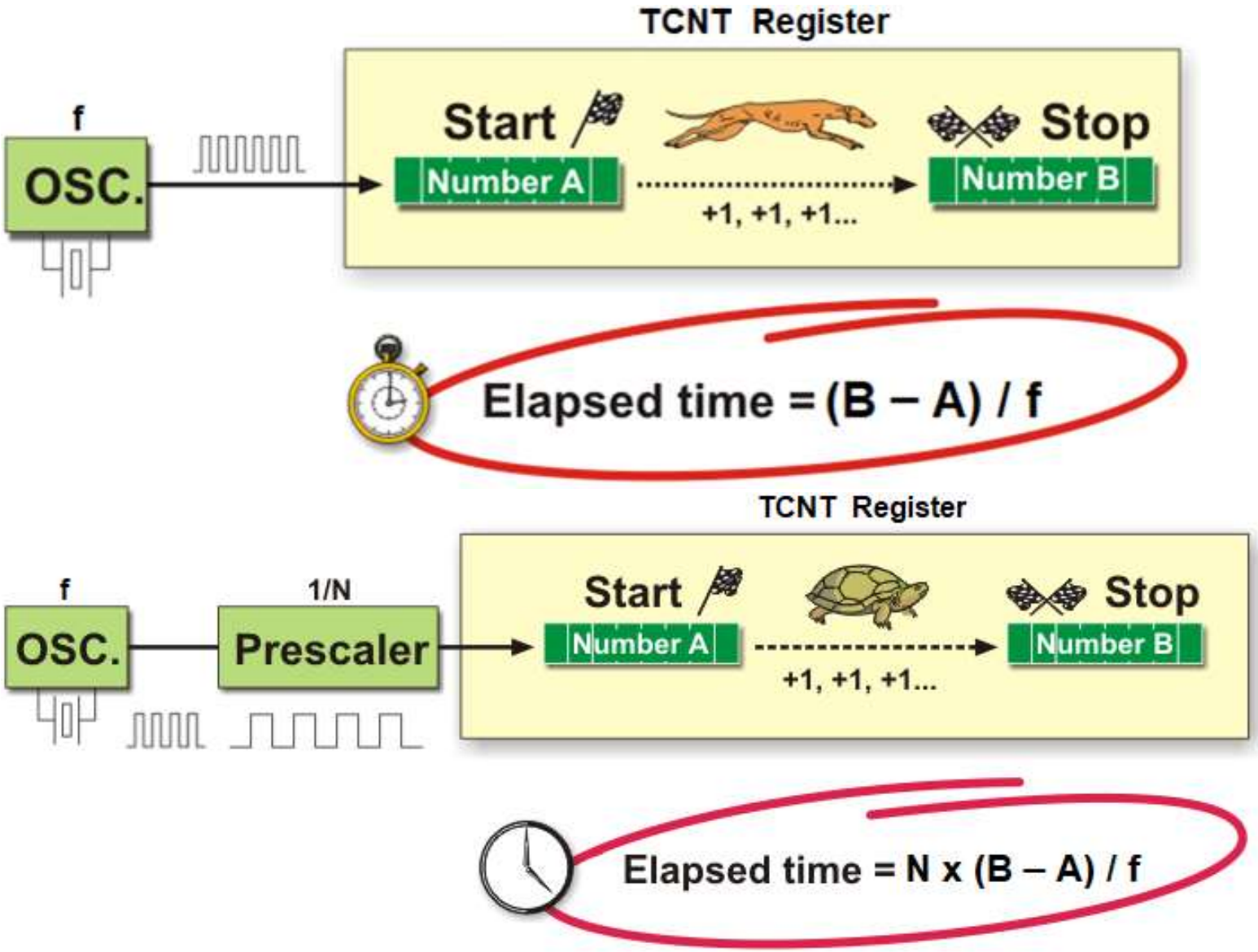


Timing and Counting

- How an 8-bit Timer Works?



Determining Elapsed Time



Timer / Counter Registers

- TCCRnB (Timer/Counter Control Register n B)



  - CSn2, CSn1 and CSn0 bits select clock source and prescale

CSn2	CSn1	CSn0	Clock Source
0	0	0	Disconnect clock source
0	0	1	F_CPU
0	1	0	F_CPU/8
0	1	1	F_CPU/64
1	0	0	F_CPU/256
1	0	1	F_CPU/1024
1	1	0	External on Tn pin, counts on falling edge
1	1	1	External on Tn pin, counts on rising edge

Timer / Counter Registers

- TCNTn (Timer/CouNTer) register of Timer n
  - Increments at every clock pulse
  - Counts up to maximum, overflows to 0 and starts again
  - Used to set the start count and also to get the end count

Timer / Counter Registers

- TIFRn (Timer Interrupt Flag Register) of Timer n



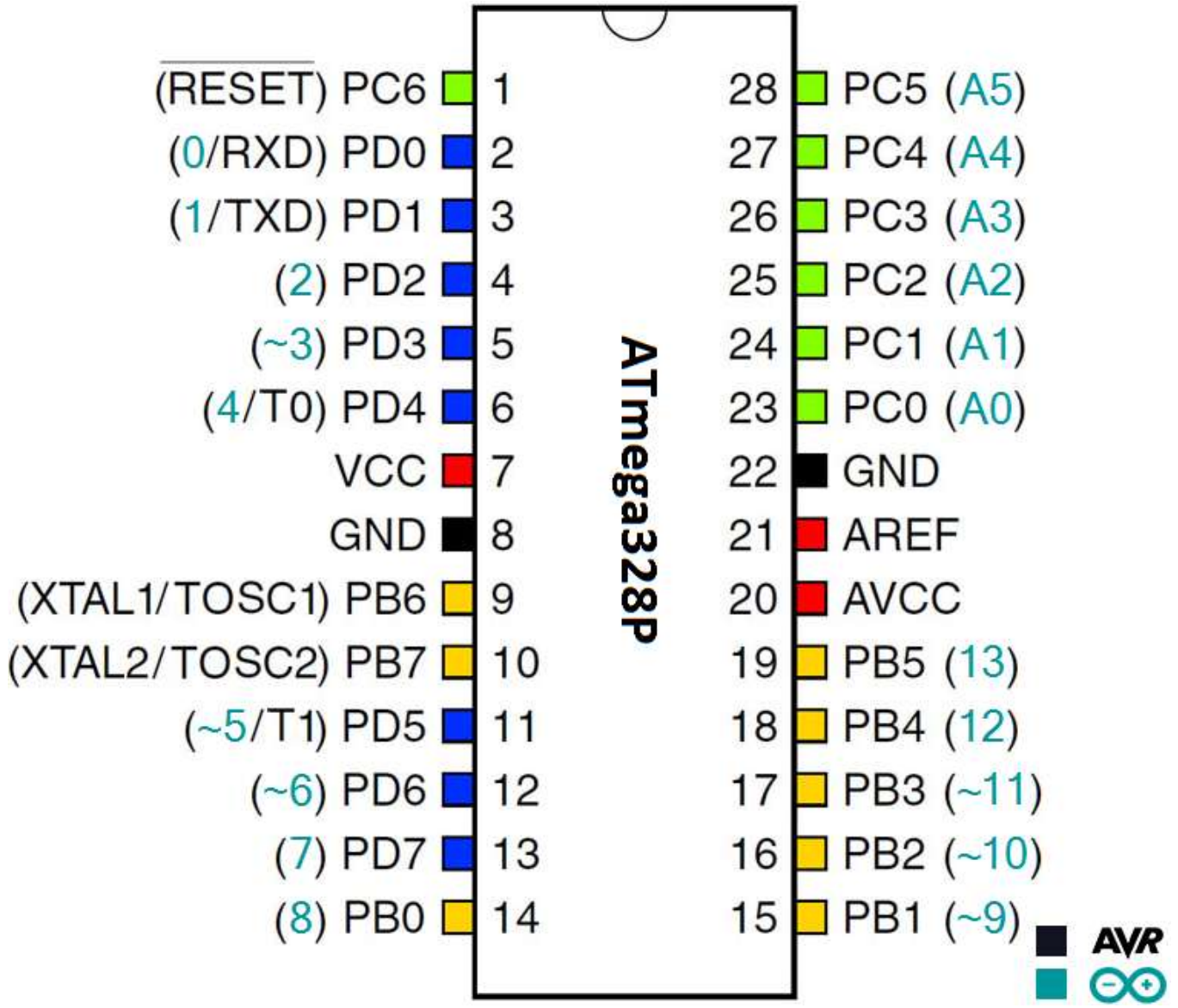
TIFRn	–	–	–	–	–	n/a	n/a	TOVn	Bit name
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	

  - when TCNTn register overflows from its maximum value to 0, TOVn (Timer Overflow n) flag bit is set
  - Set TOVn flag bit to clear it (W1C)
    - Why set it? Why not clear to clear it?

System 10

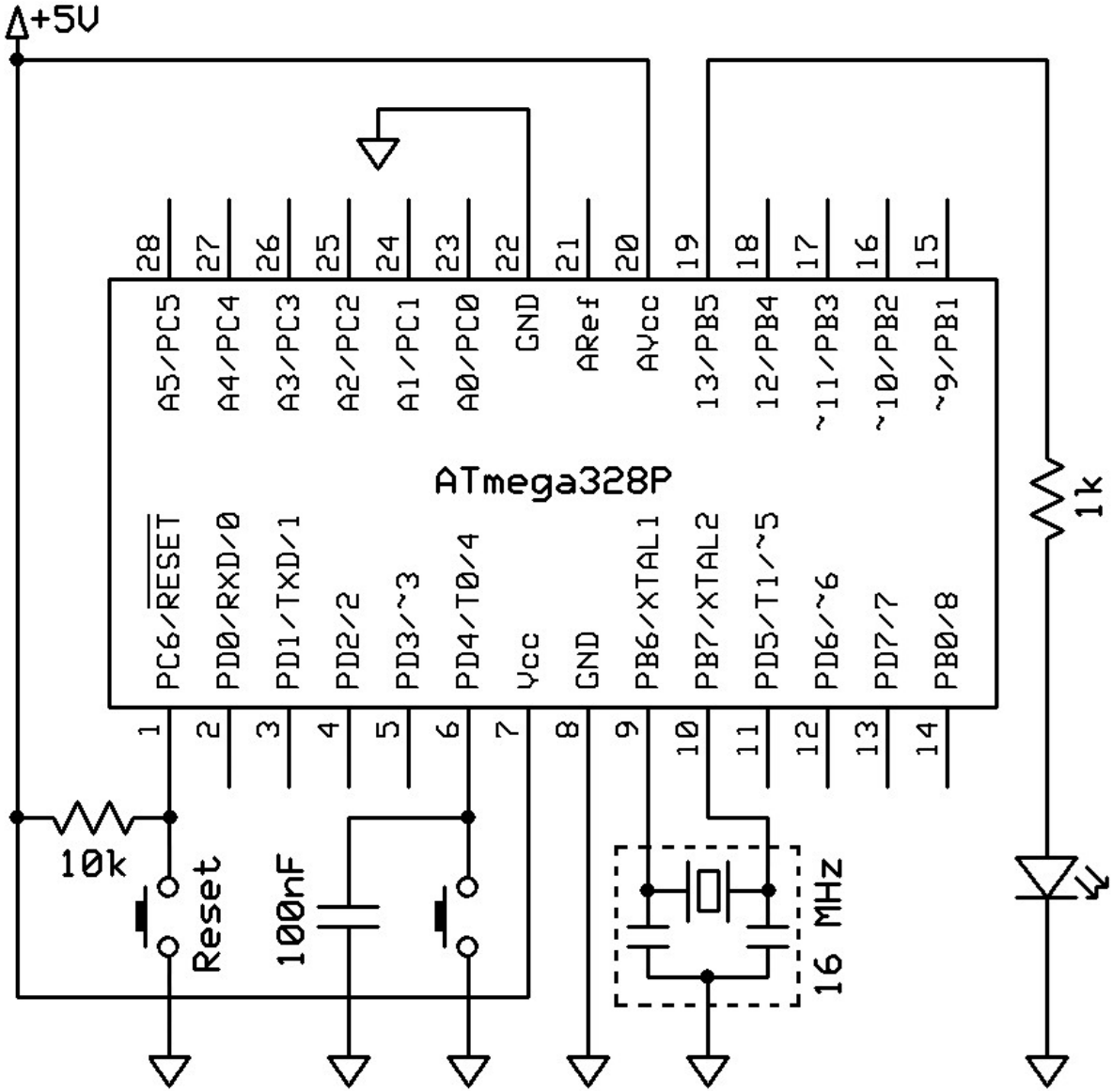
counterBasedTrigger - Counts 5 button presses and toggles an LED

ATmega328P Timer Pinout



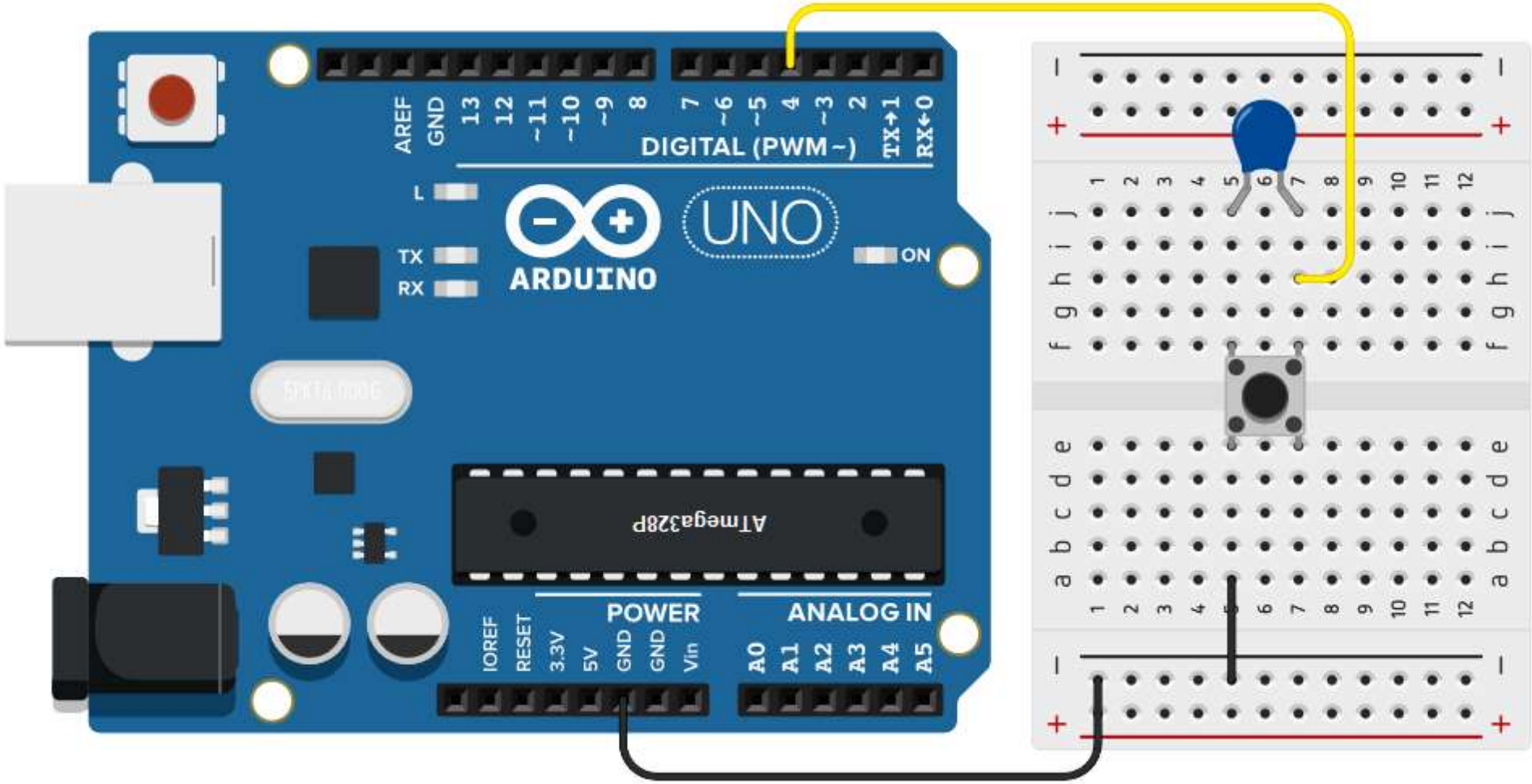
Hardware

- Schematic Diagram



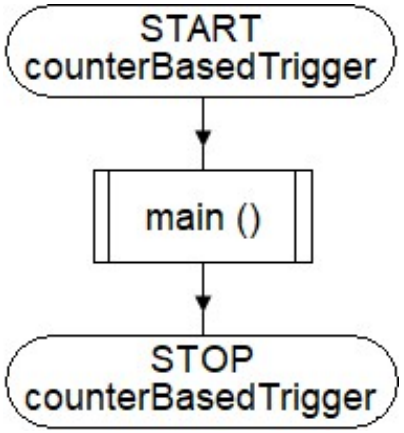
Hardware

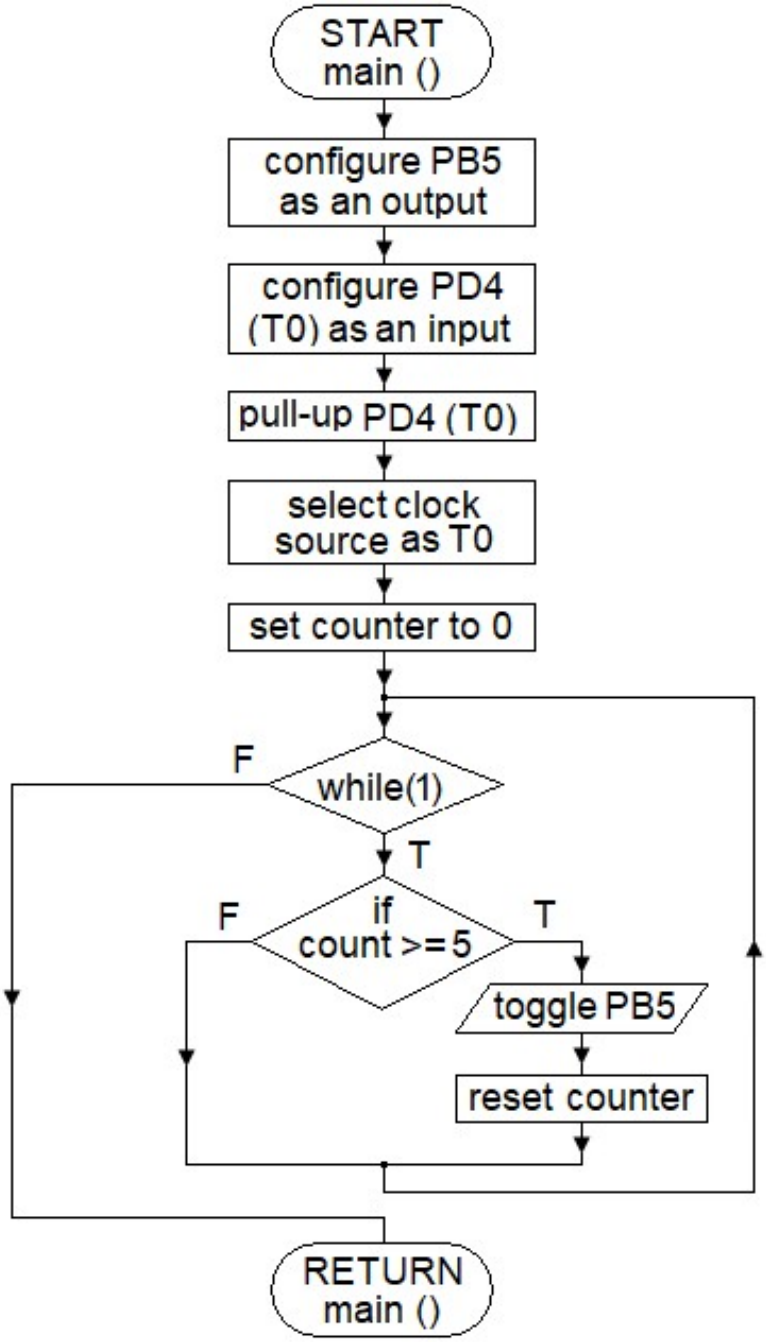
- Wiring Diagram



Firmware

- Algorithm





C Program

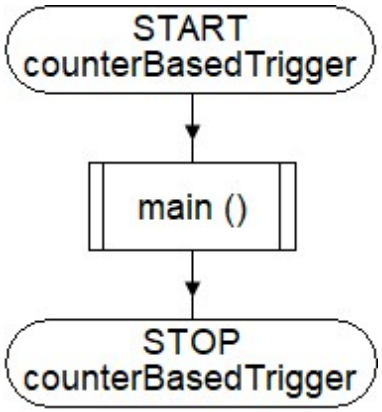
```
1   // counterBasedTriggerC.c
2
3   #include <avr/io.h>
4
5   int main(void)
6   {
7       DDRB = DDRB | 0b00100000;              // configure PB5 as an output
8       DDRD = DDRD & 0b11101111;              // configure PD4 as an input
9       PORTD = PORTD | 0b00010000;            // pull-up PD4
10      TCCR0B = TCCR0B | 0b00000110;          // select clock source as T0
11      TCNT0 = 0;                             // set counter to 0
12      while (1)
13      {
14          if (TCNT0 >= 5)                    // if counter count >= 5
15          {
16              PORTB = PORTB ^ 0b00100000;    // toggle state of PB5
17              TCNT0 = 0;                     // reset counter to 0
18          }
19      }
20      return 0;
21  }
```

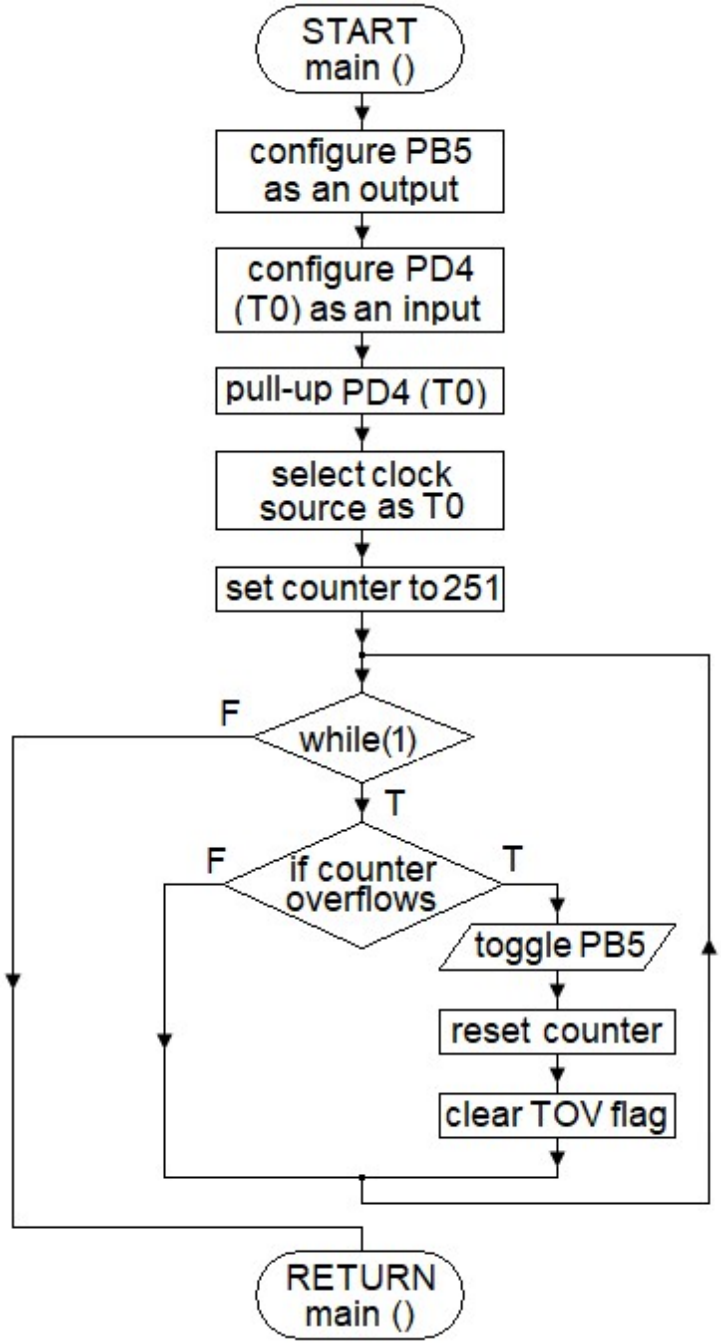
System 11

counterBasedTrigger - Counts 5 button presses using an alternate approach and toggles an LED

Firmware

- Algorithm





C Program

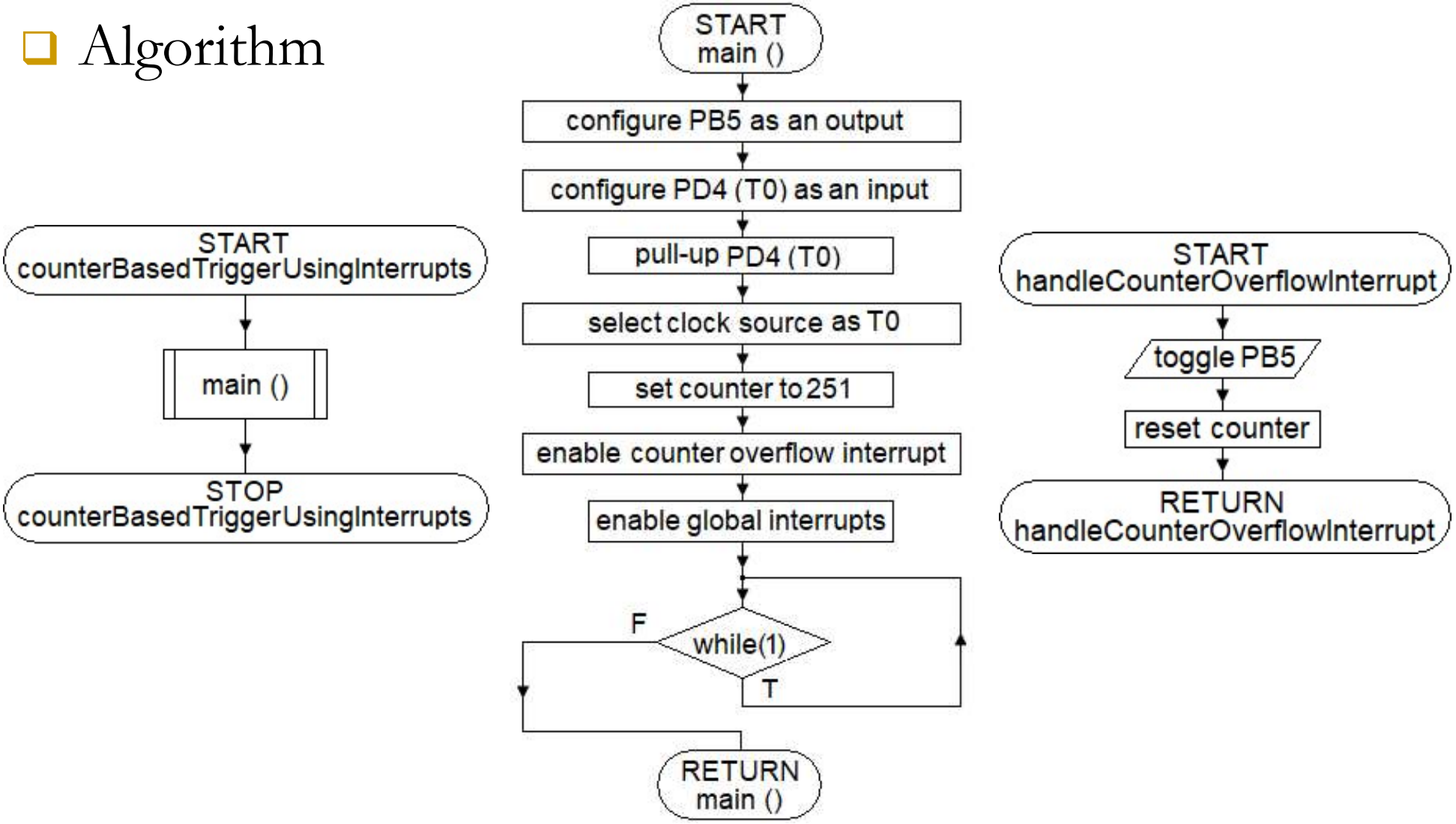
```
1   // counterBasedTriggerC.c
2
3   #include <avr/io.h>
4
5   int main(void)
6   {
7       DDRB = DDRB | 0b00100000;          // configure PB5 as an output
8       DDRD = DDRD & 0b11101111;          // configure PD4 as an input
9       PORTD = PORTD | 0b00010000;        // pull-up PD4
10      TCCR0B = TCCR0B | 0b00000110;      // select clock source as T0
11      TCNT0 = 251;                       // set counter to 251
12      while (1)
13      {
14          if (TIFR0 & 0b00000001)        // if counter overflows
15          {
16              PORTB = PORTB ^ 0b00100000;  // toggle state of PB5
17              TCNT0 = 251;               // reset counter to 251
18              TIFR0 = TIFR0 | 0b00000001;  // clear TOV0 flag
19          }
20      }
21      return 0;
22  }
```

System 12

counterBasedTriggerUsingInterrupts - Counts 5 button presses and toggles an LED using interrupts

Firmware

- Algorithm



Timer / Counter Registers

- TIMSKn (Timer Interrupt MaSK Register) of Timer n



  - TOIEn (Timer Overflow Interrupt Enable n) bit enables (or disables) Timer overflow interrupt

C Program

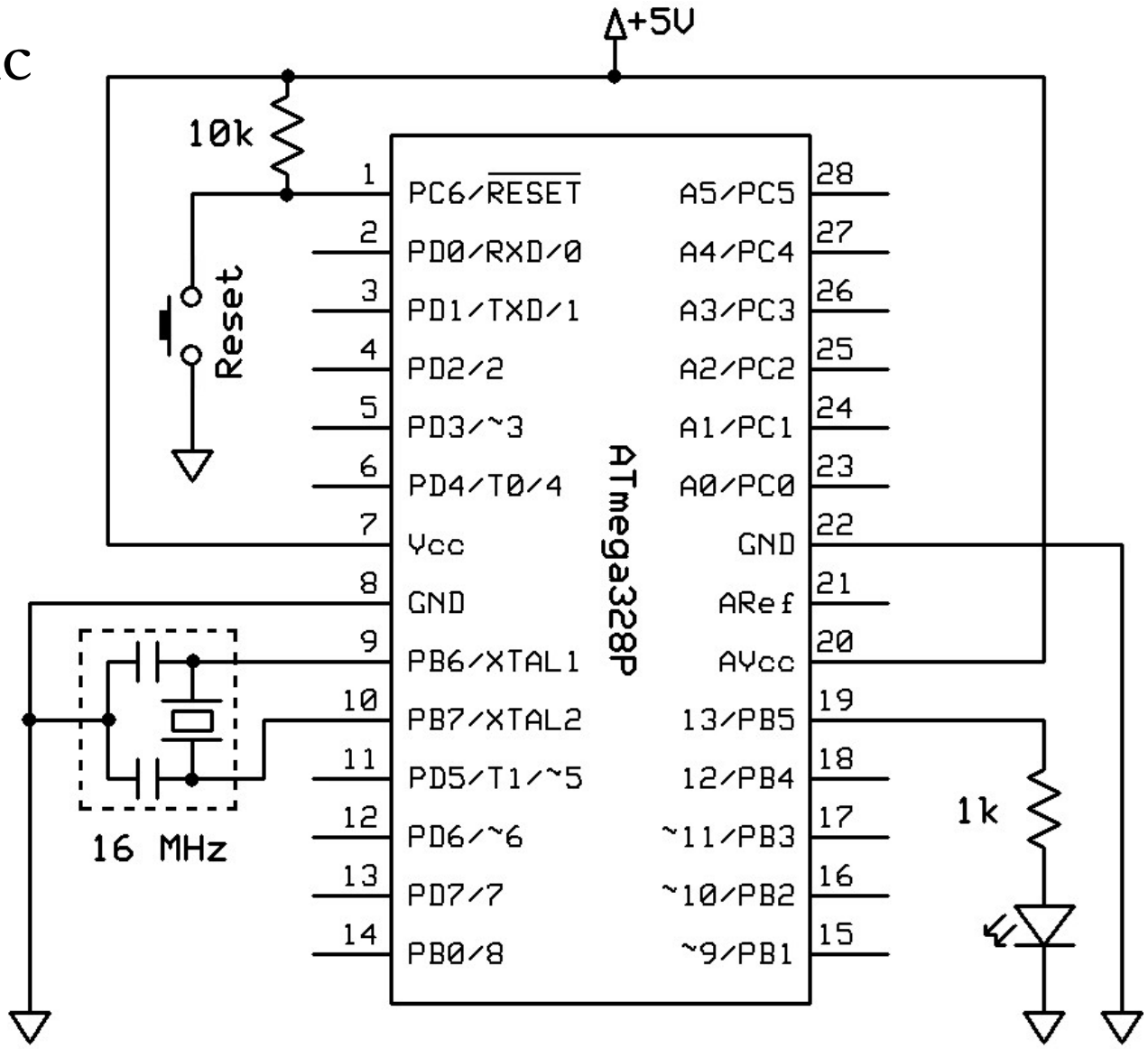
```
1   // counterBasedTriggerUsingInterruptsC.c
2
3   #include <avr/io.h>
4   #include <avr/interrupt.h>
5
6   int main(void)
7   {
8       DDRB = DDRB | 0b00100000;              // configure PB5 as an output
9       DDRD = DDRD & 0b11101111;              // configure PD4 as an input
10      PORTD = PORTD | 0b00010000;            // pull-up PD4
11      TCCR0B = TCCR0B | 0b00000110;          // select clock source as T0
12      TCNT0 = 251;                           // set counter to 251
13      TIMSK0 = TIMSK0 | 0b00000001;          // enable counter overflow interrupt
14      sei();                                 // enable global interrupts
15      while (1)
16      {                                      // can be used to perform another task
17      }
18      return 0;
19  }
20
21  ISR(TIMER0_OVF_vect)
22  {
23     PORTB = PORTB ^ 0b00100000;             // toggle state of PB5
24     TCNT0 = 251;                            // reset counter to 251
25  }
```

System 13

timerBasedBlinker - Blinks an LED in every 200 milliseconds with a Timer

Hardware

- Schematic Diagram



Calculating Timer Register Value

- Time for one clock pulse

$$time = \frac{1}{(F_CPU/prescale)} = \frac{prescale}{F_CPU}$$

- If required delay can be achieved with n clock pulses

$$delay = time \times n = \left(\frac{prescale}{F_CPU}\right) \times n$$

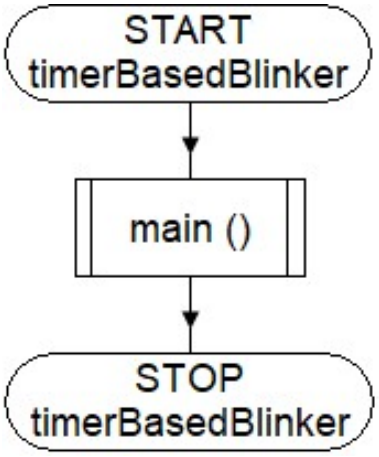
$$n = \left(\frac{F_CPU}{prescale}\right) \times delay$$

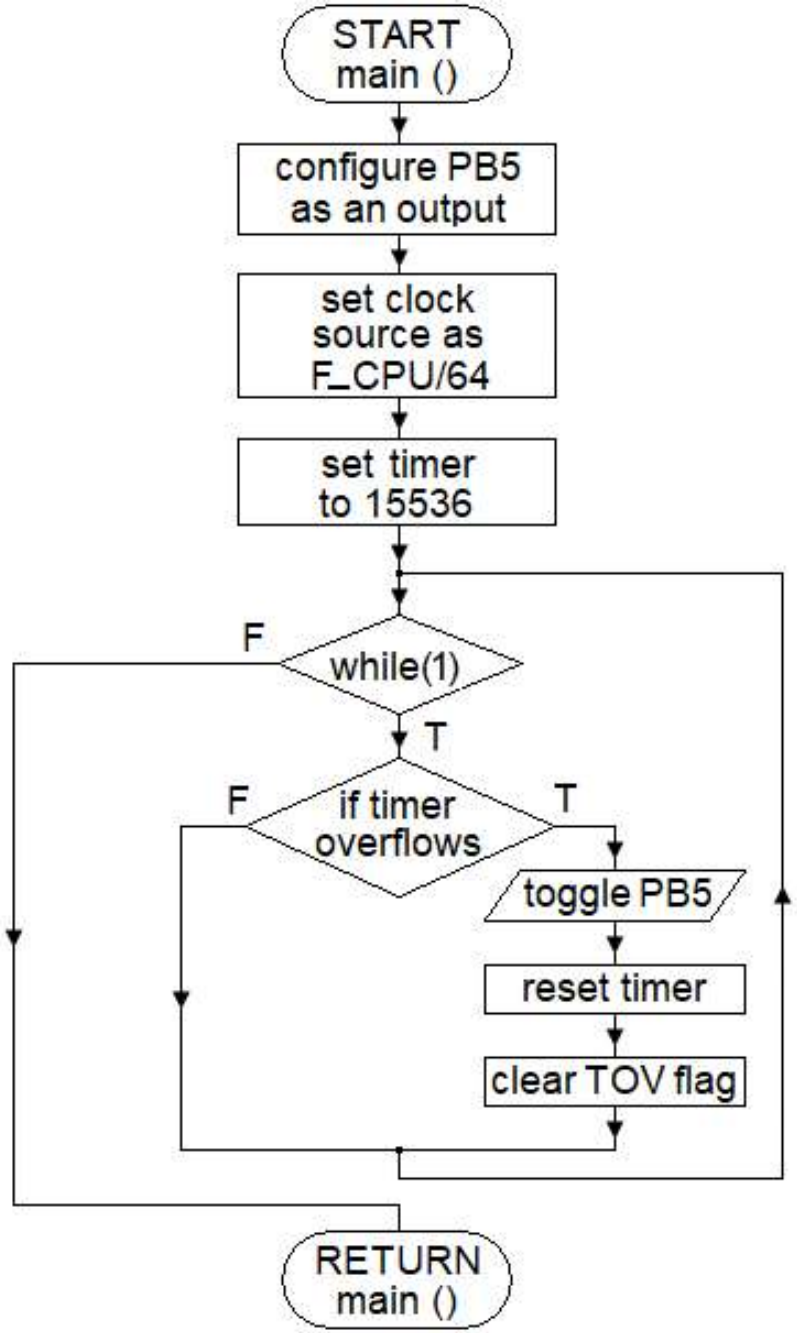
- Initial value of Timer

$$initialValue = maxValue - n + 1$$

Firmware

- Algorithm





C Program

```
1   // timerBasedBlinkerC.c
2
3   #include <avr/io.h>
4
5   int main(void)
6   {
7       DDRB = DDRB | 0b00100000;              // configure PB5 as an output
8       TCCR1B = TCCR1B | 0b00000011;          // set clock source as F_CPU/64
9       TCNT1 = 15536;                         // set timer to 15536
10      while (1)
11      {
12          if (TIFR1 & 0b00000001)            // if timer overflows
13          {
14              PORTB = PORTB ^ 0b00100000;    // toggle state of PB5
15              TCNT1 = 15536;                 // reset timer to 15536
16              TIFR1 = TIFR1 | 0b00000001;    // clear TOV1 flag
17          }
18      }
19      return 0;
20  }
```

Using Labeled Constants

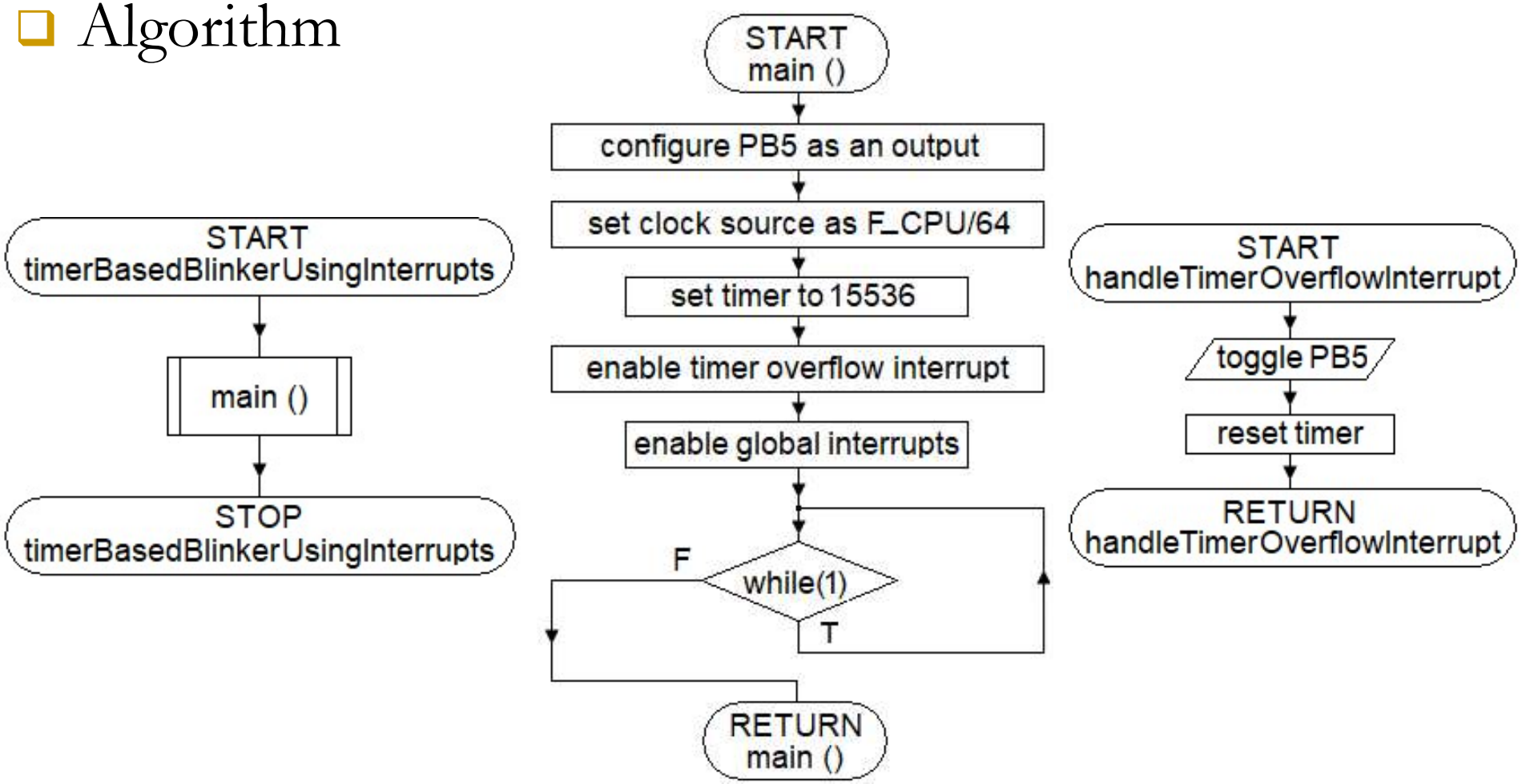
```
1   // timerBasedBlinkerC.c
2
3   #include <avr/io.h>
4
5   int main(void)
6   {
7       DDRB = DDRB | (1<<DDB5);                    // configure PB5 as an output
8       TCCR1B = TCCR1B | (1<<CS11) | (1<<CS10);// set clock source as F_CPU/64
9       TCNT1 = 15536;                              // set timer to 15536
10      while (1)
11      {
12          if (TIFR1 & (1<<TOV1))                  // if timer overflows
13          {
14              PORTB = PORTB ^ (1<<PORTB5);        // toggle state of PB5
15              TCNT1 = 15536;                      // reset timer to 15536
16              TIFR1 = TIFR1 | (1<<TOV1);          // clear TOV1 flag
17          }
18      }
19      return 0;
20  }
```

System 14

timerBasedBlinkerUsingInterrupts - Blinks an LED in every 200 milliseconds with a Timer using interrupts

Firmware

- Algorithm



C Program

```
1   // timerBasedBlinkerUsingInterruptsC.c
2
3   #include <avr/io.h>
4   #include <avr/interrupt.h>
5
6   int main(void)
7   {
8       DDRB = DDRB | 0b00100000;          // configure PB5 as an output
9       TCCR1B = TCCR1B | 0b00000011;      // set clock source as F_CPU/64
10      TCNT1 = 15536;                     // set timer to 15536
11      TIMSK1 = TIMSK1 | 0b00000001;      // enable timer overflow interrupt
12      sei();                             // enable global interrupts
13      while (1)
14      {                                  // use to perform another task
15      }
16      return 0;
17  }
18
19  ISR(TIMER1_OVF_vect)
20  {
21      PORTB = PORTB ^ 0b00100000;        // toggle state of PB5
22      TCNT1 = 15536;                     // reset timer to 15536
23  }
```

Using Labeled Constants

```
1   // timerBasedBlinkerUsingInterruptsC.c
2
3   #include <avr/io.h>
4   #include <avr/interrupt.h>
5
6   int main(void)
7   {
8       DDRB = DDRB | (1<<DDB5);                    // configure PB5 as an output
9       TCCR1B = TCCR1B | (1<<CS11) | (1<<CS10);// set clock source as F_CPU/64
10      TCNT1 = 15536;                              // set timer to 15536
11      TIMSK1 = TIMSK1 | (1<<TOIE1);               // enable timer overflow interrupt
12      sei();                                      // enable global interrupts
13      while (1)
14      {                                           // use to perform another task
15      }
16      return 0;
17  }
18
19  ISR(TIMER1_OVF_vect)
20  {
21      PORTB = PORTB ^ (1<<PORTB5);                // toggle state of PB5
22      TCNT1 = 15536;                              // reset timer to 15536
23  }
```

Questions